# UVM Verification of a Synchronous FIFO

Digital Verification using SystemVerilog and UVM - Final Project Report

Prepared by: Abdelrahman Ehab

# 1. Introduction and design under test

This report documents a complete UVM verification environment for a synchronous FIFO. The environment drives randomized and directed stimulus, checks the read data against a reference model, collects functional coverage, and uses bound SystemVerilog assertions to check the flags and internal pointers. Six RTL bugs were found and fixed; the bug report in section 7 shows the results before and after the fixes.

## 1.1 Design parameters and ports

| Signal | Direction | Description |
|---|---|---|
| clk | input | Clock (generated in FIFO_top) |
| rst_n | input | Active-low asynchronous reset |
| wr_en, rd_en | input | Write and read requests |
| data_in [15:0] | input | Write data (FIFO_WIDTH = 16) |
| data_out [15:0] | output | Registered read data |
| wr_ack | output | Registered; high one cycle after an accepted write |
| overflow | output | Registered; high one cycle after a write to a full FIFO |
| underflow | output | Registered; high one cycle after a read from an empty FIFO |
| full, empty | output | Combinational from count (count == 8, count == 0) |
| almostfull, almostempty | output | Combinational from count (count == 7, count == 1) |

FIFO depth is 8. Internal state is a memory array, a write pointer, a read pointer and a count register. A write is accepted when count is below the depth, and a read is accepted when count is not zero.

# 2. Verification plan

Each requirement has a label, the stimulus that exercises it, the functional coverage that proves it was exercised, and the check that proves it behaved correctly. A "-" in the coverage column means the item is covered by checks only.

| Label | Design Requirement Description | Stimulus Generation | Functional Coverage | Functionality Check |
|---|---|---|---|---|
| FIFO_1 | When reset is asserted, the write pointer, read pointer and count are cleared, so the FIFO is empty (empty = 1, full = 0) | Directed at the start of the simulation (FIFO_reset_seq), plus random resets in FIFO_fully_randomized_seq | - | Reset_Behaviour_p assertion; the scoreboard clears its reference queue on reset |
| FIFO_2 | When a write is requested and the FIFO is not full, the data is stored, count increments and wr_ack is asserted in the next cycle | FIFO_write_only_seq (wr_en = 1, rd_en = 0, rst_n = 1, random data_in); also FIFO_fully_randomized_seq | write_enable and wr_ack coverpoints; wr_ack_cc cross (wr_en, rd_en, wr_ack) | Write_Acknowledge_p assertion; the scoreboard reference queue stores the data for later comparison |
| FIFO_3 | When a write is requested and the FIFO is full, the write is ignored (no data overwritten) and overflow is asserted in the next cycle | FIFO_write_only_seq with more writes than the FIFO depth (10 writes, depth 8); also FIFO_fully_randomized_seq | overflow coverpoint; overflow_cc cross (wr_en, rd_en, overflow) | Overflow_Detection_p assertion; the scoreboard does not push the rejected write, so later reads would mismatch if it were stored |
| FIFO_4 | When a read is requested and the FIFO is not empty, data_out takes the oldest stored data on the next clock edge and count decrements | FIFO_read_only_seq (wr_en = 0, rd_en = 1, rst_n = 1) after the FIFO is filled; also FIFO_read_write_seq and FIFO_fully_randomized_seq | read_enable coverpoint; empty_cc cross | Scoreboard compares data_out against the reference model on every valid read |
| FIFO_5 | When a read is requested and the FIFO is empty, the read is ignored and underflow is asserted in the next cycle | FIFO_read_only_seq with more reads than stored entries (10 reads after 8 stored); also FIFO_fully_randomized_seq | underflow coverpoint; underflow_cc cross (wr_en, rd_en, underflow) | Underflow_Detection_p assertion; the scoreboard performs no read when its queue is empty |
| FIFO_6 | When read and write are requested together and the FIFO is neither full nor empty, both operations complete and count stays the same | FIFO_read_write_seq (wr_en = 1, rd_en = 1, 100 items); also FIFO_fully_randomized_seq | write_enable x read_enable crosses (wr_ack_cc, almostempty_cc, almostfull_cc) | Scoreboard (data_out checked on the read, write stored); Pointer_threshold_p assertion |
| FIFO_7 | When read and write are requested together and the FIFO is empty, the write is accepted, the read is ignored (underflow) and count increments | FIFO_read_write_seq (starts from an empty FIFO after FIFO_read_only_seq); also FIFO_fully_randomized_seq | empty_cc cross (wr_en, rd_en, empty); underflow_cc cross | Write_Acknowledge_p and Underflow_Detection_p assertions; the scoreboard pushes the write and skips the read |
| FIFO_8 | When read and write are requested together and the FIFO is full, the read is accepted, the write is rejected (overflow) and count decrements | FIFO_fully_randomized_seq (random wr_en/rd_en with the FIFO filled by write-heavy traffic) | full_cc cross (wr_en, rd_en, full); overflow_cc cross | Overflow_Detection_p assertion; the scoreboard checks data_out on the read and drops the write |
| FIFO_9 | The full flag is asserted whenever the number of stored entries equals the FIFO depth | FIFO_write_only_seq (fills the FIFO); also FIFO_fully_randomized_seq | full coverpoint; full_cc cross | Full_Flag_Assertion_p assertion |
| FIFO_10 | The empty flag is asserted whenever the FIFO holds no entries | FIFO_reset_seq, FIFO_read_only_seq (drains the FIFO); also FIFO_fully_randomized_seq | empty coverpoint; empty_cc cross | Empty_Flag_Assertion_p assertion |
| FIFO_11 | The almostfull flag is asserted when the FIFO holds depth - 1 entries | FIFO_write_only_seq; also FIFO_read_write_seq and FIFO_fully_randomized_seq | almostfull coverpoint; almostfull_cc cross | Almost_Full_Condition_p assertion |
| FIFO_12 | The almostempty flag is asserted when the FIFO holds exactly 1 entry | FIFO_read_only_seq (draining), FIFO_read_write_seq; also FIFO_fully_randomized_seq | almostempty coverpoint; almostempty_cc cross | Almost_Empty_Condition_p assertion |
| FIFO_13 | The write and read pointers wrap back to 0 after reaching the last memory location (depth - 1) | FIFO_read_write_seq and FIFO_fully_randomized_seq (long traffic that cycles through all locations) | - | Pointer_Wraparound_wr_p and Pointer_Wraparound_rd_p assertions |
| FIFO_14 | The write pointer, read pointer and count always stay within their legal ranges | FIFO_fully_randomized_seq (10000 random items, including random resets) | - | Pointer_threshold_p assertion (checked on every clock edge) |

| Label | Design Requirement Description | Stimulus Generation | Functional Coverage | Functionality Check |
|---|---|---|---|---|
| FIFO_15 | Data is read out in the same order it was written (first in, first out) with no corruption, including after pointer wraparound and reset | Random data_in in all sequences; FIFO_fully_randomized_seq for long mixed traffic | - | Scoreboard reference model compares data_out against the expected value on every valid read |

# 3. UVM testbench structure and operation

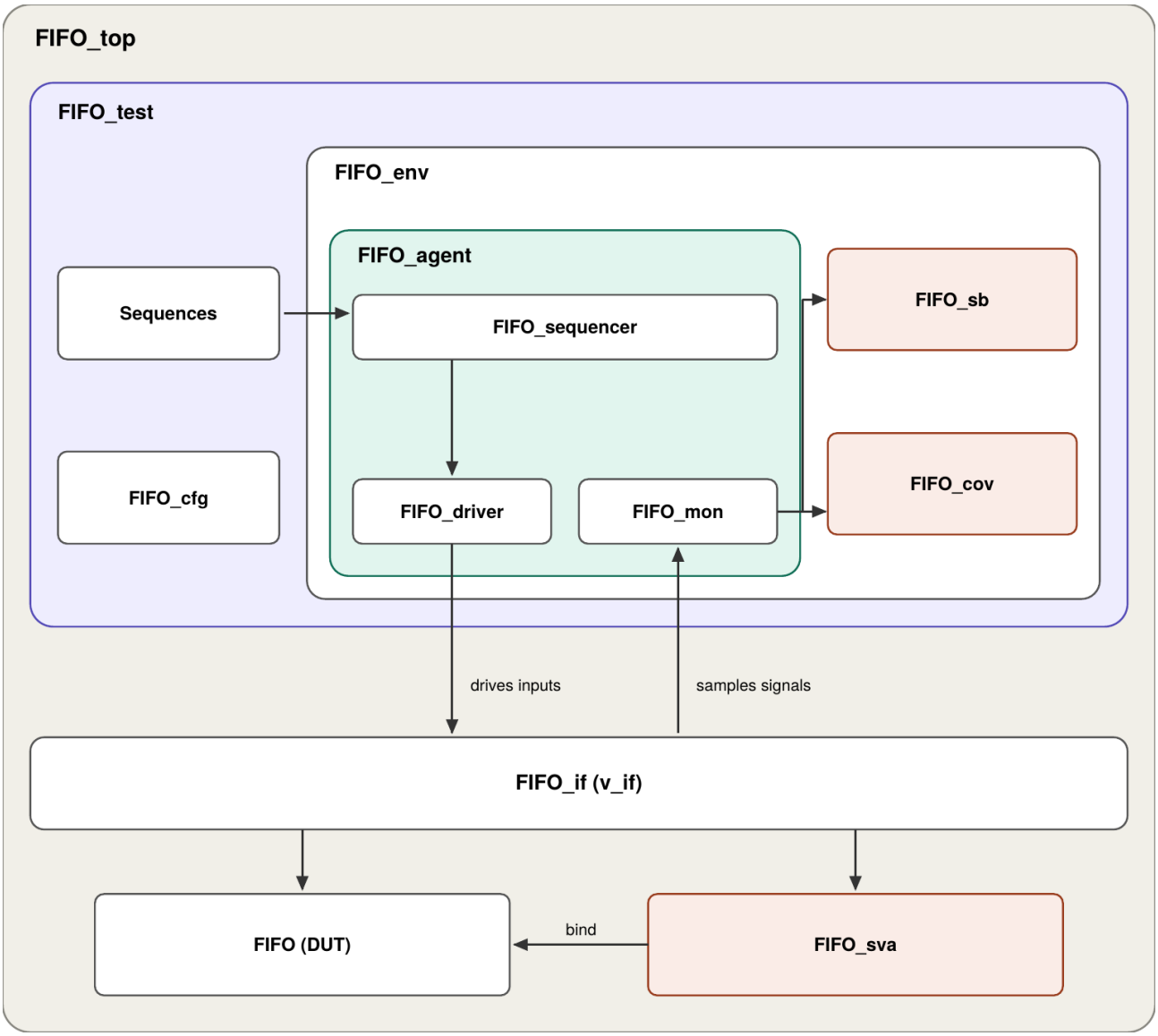



Figure 1: UVM testbench structure. Teal marks the agent, coral marks the checkers.

## 3.1 How the testbench works

**Top module (FIFO_top).** The top module generates the clock, instantiates the interface `FIFO_if` and the DUT, and binds the assertion module `FIFO_sva` into the DUT. The bind connects the internal signals wr_ptr, rd_ptr and count to the assertion module. The top then stores the virtual interface in `uvm_config_db` and calls `run_test("FIFO_test")`.

**Test (FIFO_test).** The test builds the environment and the configuration object FIFO_cfg, retrieves the virtual interface from config_db and stores it in the configuration object, which the agent passes to the driver and monitor. In the run phase it raises an objection and starts the sequences on the agent sequencer one after the other: reset, write only, read only, read and write together, then the fully randomized sequence.

**Sequence item (FIFO_seq_item).** The item holds the randomized inputs (data_in, rst_n, wr_en, rd_en) and the observed outputs (data_out and the flags). Constraints keep reset rare (3% low) and bias the enables (write 70% high, read 30% high) so that the FIFO fills up and the overflow cases are reached, while the directed sequences override the constraints with inline `randomize() with`.

**Driving.** The sequencer hands each item to FIFO_driver through `get_next_item()`. The driver drives only the DUT inputs (rst_n, data_in, rd_en, wr_en) onto the virtual interface, waits for the next negedge of the clock, and calls `item_done()`.

**Monitoring.** FIFO_mon samples all interface signals, inputs and outputs, at every negedge into a new sequence item and writes it to its analysis port. The agent forwards it through `agt_ap`, and the environment connects `agt_ap` to both the scoreboard and the coverage collector.

**Analysis.** FIFO_sb contains a reference model built on a queue. On reset it clears the queue. For each item it decides whether a read happens (rd_en and the queue is not empty) and whether a write happens (wr_en and the queue is not full), both evaluated on the queue state before the item. A valid read pops the oldest word and compares it with data_out. A valid write pushes data_in. The scoreboard checks only data_out; the flags are checked by the assertions. FIFO_cov samples the covergroup g1 on every item, covering each interface signal and its crosses with wr_en and rd_en.

**Assertions.** FIFO_sva is bound inside the DUT, so it sees the interface and the internal pointers. It checks the flags, acknowledge, overflow and underflow timing, pointer wraparound and pointer limits on every clock edge, independently of the UVM components. Each assertion has a matching cover property that records how often its scenario really happened.

# 4. UVM sequences and waveform snippets

Five sequences are started by FIFO_test in the order below. Each snippet is a QuestaSim waveform of the interface signals.

## 4.1 FIFO_reset_seq

A single item with rst_n = 0, wr_en = 0, rd_en = 0 and data_in = 0, run at the start of the simulation. The driver applies the reset to the interface, which clears the pointers and the count (empty = 1) and, after the RTL fixes, the registered outputs wr_ack, overflow, underflow and data_out. The reset is visible at the start of the write-only waveform below, where rst_n goes high before the first write.

## 4.2 FIFO_write_only_seq

Ten items with wr_en = 1, rd_en = 0 and rst_n = 1, so the FIFO is filled past its depth of 8.

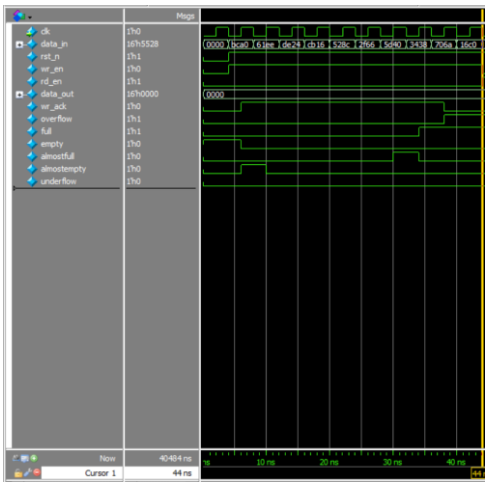


Figure 2: write-only sequence. After reset, wr_ack goes high one cycle after the first write and empty falls; almostempty pulses at one stored entry, almostfull at seven, and full goes high at eight. The extra writes are rejected: wr_ack drops and overflow rises.

## 4.3 FIFO_read_only_seq

Ten items with wr_en = 0, rd_en = 1 and rst_n = 1. This drains the 8 stored words and then reads from the empty FIFO.

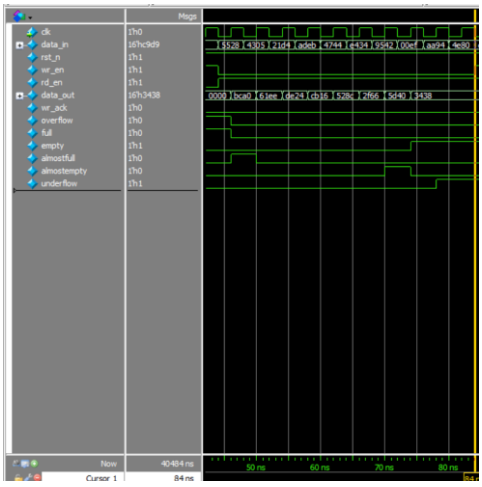


Figure 3: read-only sequence. data_out returns the stored words in the order they were written, almostfull falls, almostempty pulses at the last entry, then empty rises and underflow is asserted for the reads that find the FIFO empty.

## 4.4 FIFO_read_write_seq

One hundred items with wr_en = 1, rd_en = 1 and rst_n = 1. The sequence starts from an empty FIFO, so the first write is accepted and the simultaneous read is ignored; from then on each cycle stores one word and reads one word.

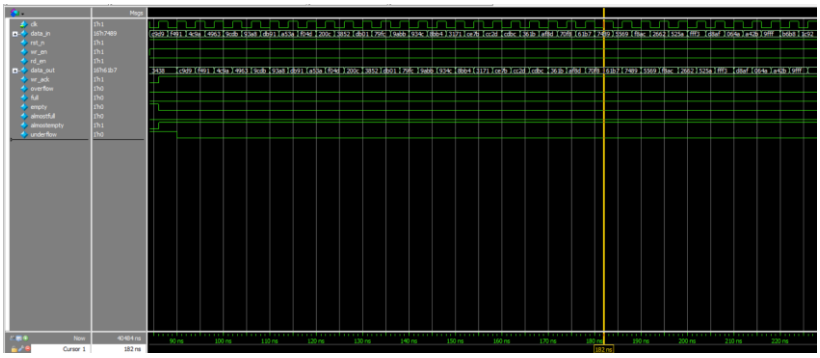


Figure 4: read-and-write sequence. wr_ack stays high and count holds at one entry (almostempty = 1). data_out follows data_in with a one-item delay, and no flag, overflow or underflow is raised.

## 4.5 FIFO_fully_randomized_seq

Ten thousand fully randomized items using the constraints in the sequence item (reset low 3%, wr_en high 70%, rd_en high 30%). This sequence provides the long mixed traffic that reaches all flags, the overflow and underflow cases, simultaneous read and write at the full and empty boundaries, pointer wraparound and random resets.

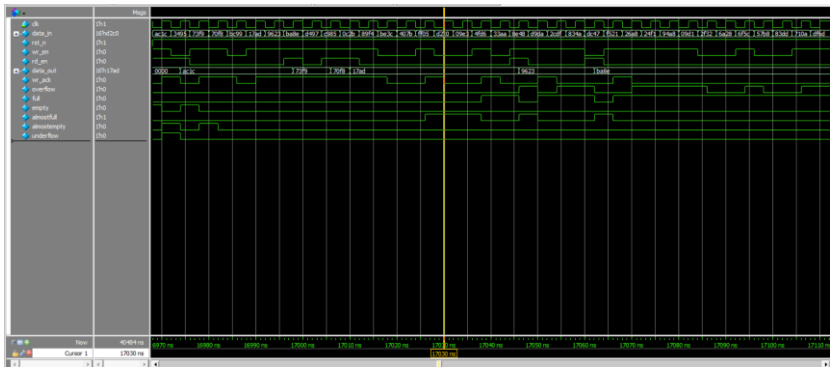


Figure 5: fully randomized sequence. The enables change randomly and the flags, wr_ack, overflow and underflow all toggle in response.

# 5. Assertions

The assertions are in FIFO_sva and are bound to the DUT in FIFO_top. Each one also has a cover property with the same name ending in _c.

| Feature | Assertion | Name |
|---|---|---|
| Reset clears the write pointer, read pointer and count | `@(posedge clk) !rst_n \|-> (wr_ptr == 0 && rd_ptr == 0 && count == 0)` | `Reset_Behaviour_p` |
| A write to a non-full FIFO raises wr_ack in the next cycle | `@(posedge clk) disable iff(!rst_n) (wr_en && !full) \|=> wr_ack` | `Write_Acknowledge_p` |
| A write to a full FIFO raises overflow in the next cycle | `@(posedge clk) disable iff(!rst_n) (wr_en && full) \|=> overflow` | `Overflow_Detection_p` |
| A read from an empty FIFO raises underflow in the next cycle | `@(posedge clk) disable iff(!rst_n) (rd_en && empty) \|=> underflow` | `Underflow_Detection_p` |
| empty is asserted when count is 0 | `@(posedge clk) (count == 0) \|-> empty` | `Empty_Flag_Assertion_p` |
| full is asserted when count equals the FIFO depth | `@(posedge clk) (count == FIFO_DEPTH) \|-> full` | `Full_Flag_Assertion_p` |
| almostfull is asserted when count is depth - 1 | `@(posedge clk) (count == FIFO_DEPTH-1) \|-> almostfull` | `Almost_Full_Condition_p` |
| almostempty is asserted when count is 1 | `@(posedge clk) (count == 1) \|-> almostempty` | `Almost_Empty_Condition_p` |
| The write pointer wraps to 0 after the last location | `@(posedge clk) disable iff(!rst_n) (wr_ptr == FIFO_DEPTH-1 && wr_en && !full) \|=> (wr_ptr == 0)` | `Pointer_Wraparound_wr_p` |
| The read pointer wraps to 0 after the last location | `@(posedge clk) disable iff(!rst_n) (rd_ptr == FIFO_DEPTH-1 && rd_en && !empty) \|=> (rd_ptr == 0)` | `Pointer_Wraparound_rd_p` |
| Pointers and count always stay in their legal range | `@(posedge clk) (rd_ptr < FIFO_DEPTH && wr_ptr < FIFO_DEPTH && count <= FIFO_DEPTH)` | `Pointer_threshold_p` |

# 6. Coverage reports

## 6.1 Code coverage

Code coverage was collected on the DUT (design unit work.FIFO) with branch, statement and toggle coverage enabled (`+cover=bst`). The full report is in code_coverage_report.txt.

| Coverage type | Bins | Hits | Misses | Coverage |
|---|---|---|---|---|
| Branch | 28 | 28 | 0 | 100.00% |
| Statement | 30 | 30 | 0 | 100.00% |
| Toggle | 20 | 20 | 0 | 100.00% |

Every branch and statement of the RTL executed, and the count, rd_ptr and wr_ptr registers toggled in both directions. 10121 transactions were applied in total (1 reset, 10 write-only, 10 read-only, 100 read-and-write and 10000 random).

## 6.2 Functional coverage

The covergroup g1 in FIFO_cov has 9 coverpoints (write_enable, read_enable, wr_ack, overflow, full, empty, almostfull, almostempty, underflow), each with a bin for 0 and a bin for 1, and 7 crosses of wr_en, rd_en and a flag. All 25 bins were hit, giving 100.00% (functional_coverage_report.txt).

| Coverpoint | Bin 0 hits | Bin 1 hits |
|---|---|---|
| write_enable | 2990 | 7131 |
| read_enable | 6973 | 3148 |
| wr_ack | 5557 | 4564 |
| overflow | 7751 | 2370 |
| full | 6628 | 3493 |
| empty | 9592 | 529 |
| almostfull | 7839 | 2282 |
| almostempty | 9303 | 818 |
| underflow | 9959 | 162 |

Each of the seven crosses (wr_ack_cc, overflow_cc, full_cc, empty_cc, almostfull_cc, almostempty_cc, underflow_cc) has one bin and was hit 10121 times.

## 6.3 Sequential domain coverage

Sequential domain coverage shows whether the temporal scenarios described by the assertions happened during simulation. The table compares the assertion coverage of FIFO_sva before and after the RTL fixes (assertion_report.txt).

| Run | FIFO_sva assertions | Hits | Misses | Coverage |
|---|---|---|---|---|
| Before RTL fixes | 11 | 9 | 2 | 81.81% |
| After RTL fixes | 11 | 11 | 0 | 100.00% |

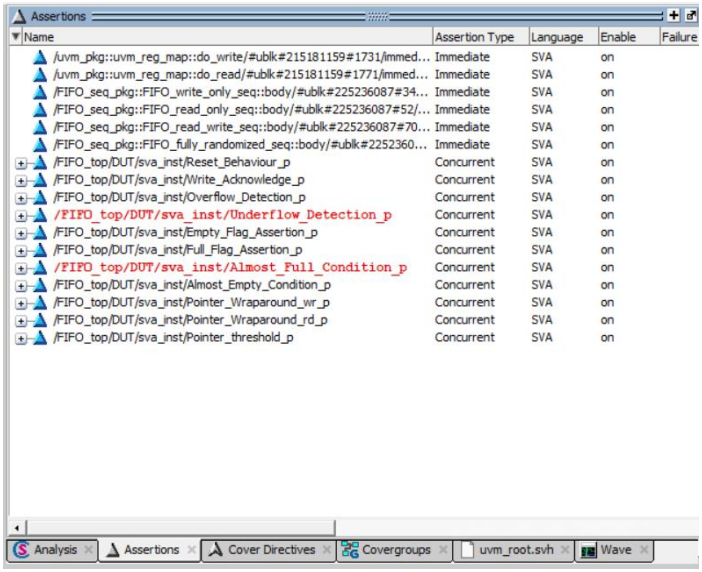



Figure 6: assertions window before the fixes. Underflow_Detection_p and Almost_Full_Condition_p fail.

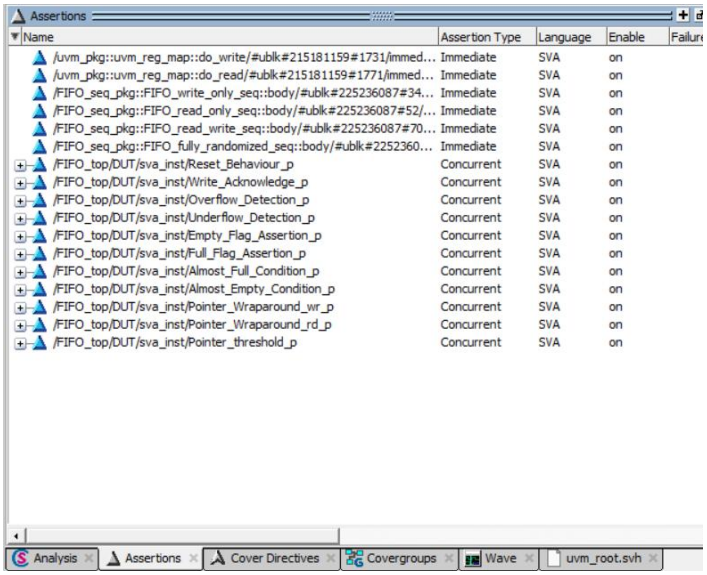



Figure 7: assertions window after the fixes. No assertion fails.

# 7. Bug report

Six bugs were found in FIFO.sv. Two were exposed by failing assertions, the others by waveform inspection and code review. The table lists each bug, how it was detected and how it was fixed.

| # | Bug | Symptom / detection | Fix |
|---|---|---|---|
| 1 | almostfull compared count with FIFO_DEPTH-2 | Almost_Full_Condition_p failed: almostfull stayed low when count reached depth - 1 | Compare with FIFO_DEPTH-1 |
| 2 | underflow was a combinational assign of (empty && rd_en), unlike the registered wr_ack and overflow | Underflow_Detection_p failed: underflow dropped as soon as rd_en dropped instead of being held one cycle after the read | Register underflow in a clocked always block |
| 3 | count was not updated when wr_en and rd_en were both high | Waveform of the read-and-write sequence: a write to an empty FIFO left empty high and reads never started; at full a read did not free a slot | Add the 2'b11 case: count + 1 when empty, count - 1 when full, unchanged otherwise |
| 4 | wr_ack and overflow were not reset | Waveform: both signals were X after reset until the first write | Reset wr_ack and overflow to 0 in the reset branch |
| 5 | overflow was only updated in the final else branch, so it stayed high after a later accepted write | Code review of the write block | Clear overflow when a write is accepted |
| 6 | data_out was not reset | Waveform: data_out was X until the first read | Reset data_out to 0 |

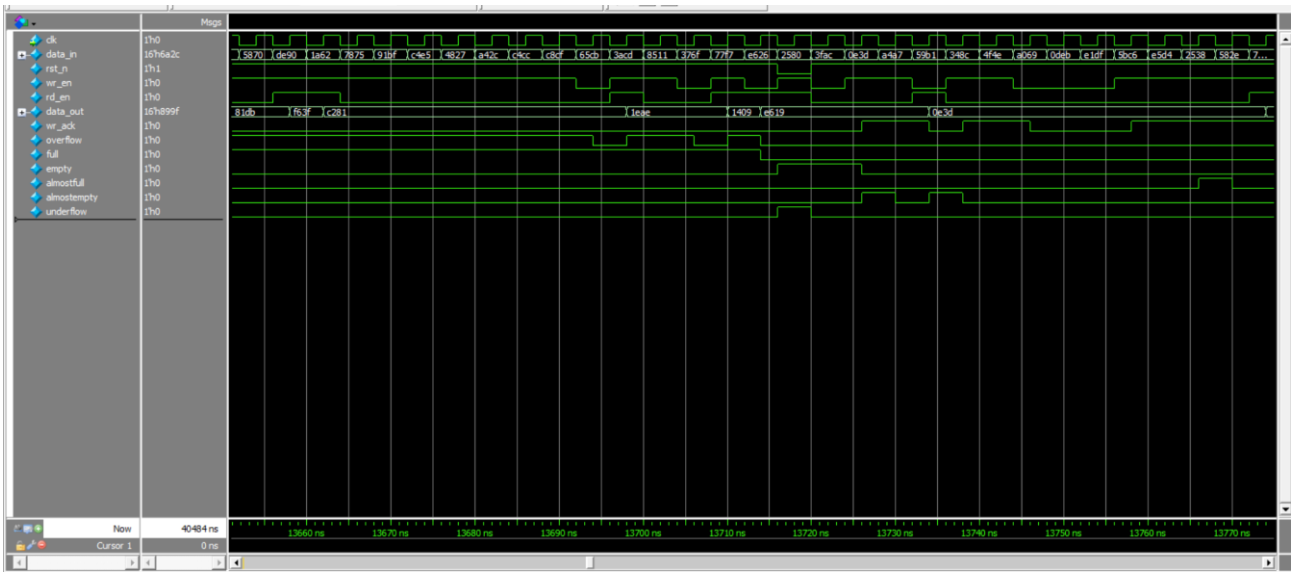



Figure 8: waveform before the fixes (read-and-write traffic). Flags and outputs do not follow the expected behavior at the empty and full boundaries.

After the fixes, no assertion fails (figure 7), assertion coverage is 100% and the code and functional coverage are 100% (section 6).